



UNIDAD 2

Algoritmo de Trayectoria.

TEMA 2:

Implementación y Aplicaciones.

Descripción:

Los algoritmos de trayectoria comprenden una familia de métodos metaheurísticos que exploran el espacio de soluciones siguiendo una única trayectoria individual, a diferencia de las técnicas que operan con poblaciones. Su mecanismo se fundamenta en la definición de vecindarios, los cuales permiten transitar de una solución candidata a otra mediante movimientos locales, con el objetivo de mejorar progresivamente el valor de una función objetivo.

Al finalizar la unidad, el estudiante estará en capacidad de comprender los fundamentos conceptuales de los algoritmos de trayectoria, implementarlos en problemas concretos y evaluar su rendimiento en comparación con otros métodos metaheurísticos.

Metaheurísticas

PERIODO ACADÉMICO 2026-1S

TABLA DE CONTENIDO

Tema 2. Implementación y Aplicaciones.....	2
2.1. Consideraciones prácticas de implementación.....	2
2.2. Diseño de funciones de costo, criterios de parada y ajuste de parámetros.....	4
2.3. Aplicaciones en logística, robótica, planificación de rutas y asignación de recursos.	10
2.4. Evaluación empírica de rendimiento frente a problemas benchmark.....	12
Bibliografía	15

Tema 2. Implementación y Aplicaciones.

2.1. Consideraciones prácticas de implementación.

La implementación efectiva de un algoritmo de trayectoria (o de búsqueda) trasciende la mera codificación, constituyendo un proceso que conlleva una serie de decisiones estratégicas que inciden directamente en su eficacia y rendimiento.

Entre estas decisiones destacan la definición del espacio de búsqueda, el diseño de una arquitectura modular, la gestión de recursos y la validación progresiva, además de la correcta configuración de sus componentes algorítmicos centrales.

Uno de los aspectos iniciales y más críticos radica en la definición del espacio de búsqueda el cual determina la totalidad de soluciones posibles. Si dicho espacio es excesivamente amplio, el algoritmo puede incurrir en ineficiencias computacionales, mientras que, si resulta demasiado restrictivo, existe el riesgo de omitir soluciones óptimas. Una vez delimitado este espacio, la implementación se sustenta en cuatro pilares conceptuales esenciales.

- **Representación de las soluciones:** Este paso consiste en definir una estructura de datos que codifique de manera eficiente una solución factible del problema. La elección de esta representación debe ser intuitiva, permitir la generación de soluciones iniciales válidas y facilitar la definición de movimientos hacia soluciones vecinas.
- **Definición de la función de vecindad($N(S)$):** Para una solución dada s , esta función genera un conjunto de soluciones $N(S)$ rechazables mediante una modificación o "movimiento" simple (intercambiar dos elementos, invertir una secuencia). El diseño de la vecindad determina la granularidad de la exploración y es

crucial para evitar quedar atrapado en óptimos locales prematuramente.

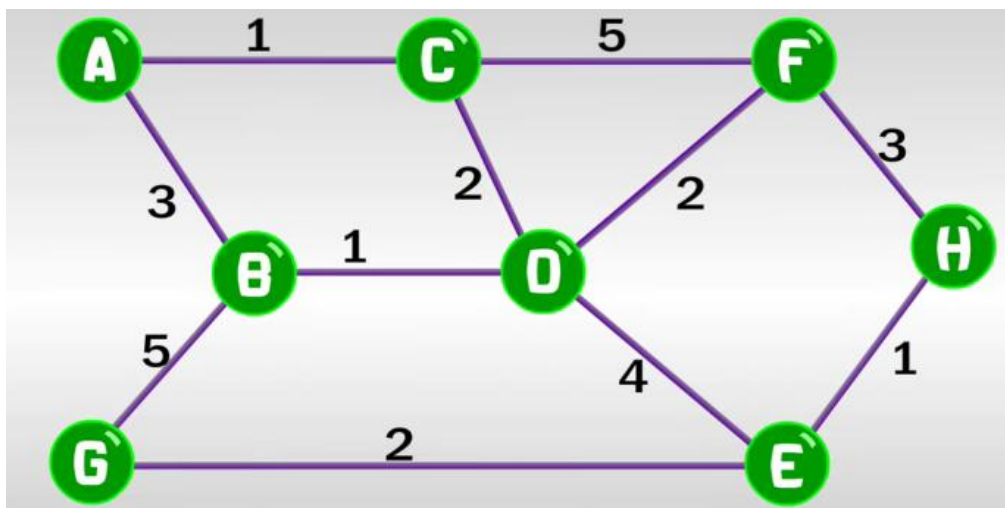
- **Evaluación mediante la Función Objetivo(s):** Esta función asigna un valor numérico a cada solución s , cuantificando su calidad. Dado que se invoca un número muy elevado de veces, su implementación debe ser extremadamente eficiente para reducir el tiempo total de ejecución.
- **Establecimiento del Criterio de Parada:** Es necesario definir las condiciones que finalizan la ejecución, como un número de iteraciones, un límite de tiempo, o un número de iteraciones sin mejora en la solución.

● Ejemplo aplicado:

Problema de Rutas en un Grafo.

Para ilustrar la aplicación de los pasos generales, considere el siguiente grafo ponderado con 8 nodos (A-H). El objetivo es encontrar una ruta de costo mínimo que conecte el nodo A con el nodo H.

Figura 1. Ejemplo de grafo ponderado para un problema de rutas



Fuente: Studios Cat [<https://www.youtube.com/watch?v=LLx0QVMZVkk>]

- **Representación de la Solución:**

Una posible solución inicial puede representarse como la secuencia de nodo [A, B, D, E H].

- **Definición de la Función de Vecindad (N(s)):**

La función objetivo se define como la suma de los pesos de las aristas de la ruta.

$$f([A, B, D, E, H]) = 3 + 1 + 4 + 1 = 9$$

$$f([A, C, D, E, H]) = 1 + 2 + 4 + 1 = 8$$

$$f([A, B, D, F, H]) = 3 + 1 + 2 + 3 = 9$$

- **Criterio de Parada:**

El proceso de exploración se detiene tras 10 iteraciones o cuando no se observa mejora en tres movimientos consecutivos.

2.2. Diseño de funciones de costo, criterios de parada y ajuste de parámetros.

El corazón de cualquier algoritmo de planificación y optimización es la función de costo (también conocido como función objetivo).

Esta función actúa como el sistema nervioso de algoritmo, pues no solo determina la calidad de una solución en comparación con otras, sino que guía activamente la búsqueda hacia regiones prometedoras del espacio de soluciones.

Por otro lado, los criterios de parada constituyen el mecanismo de control que define cuando finalizar la búsqueda equilibrando el trade-off entre el tiempo de computación y la calidad de la solución encontrada. El diseño efectivo de estos componentes es crucial, ya que incluso

algoritmos sofisticados pueden verse obstaculizados por funciones de costo mal definidas o criterios de parada inadecuados, un desafío que es particularmente relevante en problemas del mundo real con restricciones complejas (Weiss & Kaminka, 2023).

2.2.1. Diseño de la función de Costo

Una función de costo bien diseñada es crucial para el éxito del algoritmo. Debe ser una abstracción cuantitativa de los objetos y restricciones del mundo real. Una función de costo ideal debe cumplir con tres propiedades principales.

- **Relevancia:** Debe reflejar de manera fiel y completa los objetivos primarios y secundarios del problema. Una función irrelevante o incompleta puede llevar al algoritmo a encontrar soluciones técnicamente "óptimas" pero inútiles en la práctica.
- **Eficiencia:** Debe ser computacionalmente simple de evaluar. Los algoritmos de optimización evalúan esta función miles o incluso millones de veces, por lo que una función demasiado compleja puede ralentizar críticamente todo el proceso.
- **Flexibilidad:** Debe permitir incorporar, ponderar y combinar de manera sencilla múltiples restricciones o preferencias adicionales (seguridad, confort, costos variables) sin necesidad de rediseñar el algoritmo por completo.

● **Ejemplo Básico:** En el Problema del viajante (TSP), la función de costo canónico es la distancia total recorrida. Sin embargo, esta es una simplificación que no captura la complejidad de los escenarios reales.

● Ejemplos Avanzados y Multi-objetivo:

En contextos modernos como la logística o la robótica, la función de costos suele ser una combinación ponderada de múltiples factores. Por ejemplo, la función de costo $C(T)$ para una trayectoria T podría definirse como:

$$\begin{aligned} C(T) = & W_1 \cdot \text{Distancia} \\ & + W_2 \cdot \text{Tiempo de Viaje} \\ & + W_3 \cdot \text{Consumo de Combustible} \\ & + W_4 \cdot \text{Emisión de CO}_2 \\ & + W_5 \cdot \text{Penalización por Retraso} \\ & + W_6 \cdot \text{Riesgo en la Trayectoria} \end{aligned}$$

Donde los pesos son $W_1, W_2, \dots, W_n$ asignan prioridad relativa a cada objetivo. La elección de estos pesos es en sí misma un subproblema de diseño y puede realizarse mediante análisis de sensibilidad o técnicas de optimización multi-objetivo.

Técnicas de Diseño:

- **Normalización:** cuando los factores tienen unidades y escalas diferentes (distancia en km vs riesgo adimensional), esencial normalizarlos (escalando entre 0 y 1) para que la suma ponderada sea meaningful y un factor no domine a los otros por simple magnitud numérica.
- **Funciones de Penalización:** Es una metodología robusta para manejar restricciones. Se añade un término grande a la función de costo cada vez que una solución violenta una restricción (chocar con un obstáculo, exceder el tiempo máximo de viaje). Esto obliga al algoritmo a evitar regiones inviables.
- **Aprendizaje por Refuerzo (Inverso):** En aplicaciones avanzadas, la función de costo puede no ser explícita. Técnicas de aprendizaje por refuerzo inverso pueden inferir una función de costo a partir de

demostraciones expertas (observando qué trayectoria elige un operador humano).

2.2.2. Definición por Criterios de Parada.

Los Criterios de parada son esenciales para evitar que el algoritmo se ejecute indefinidamente, especialmente cuando se busca el óptimo global en problemas NP-difíciles que son intrínsecamente complejos. La elección del criterio o la combinación de varios define el balance entre explotación (profundizar en una buena solución) y exploración (buscar nuevas soluciones).

Entre los criterios más utilizados se encuentran:

- **Iteraciones Máximas (Número de Evaluaciones):** Se ejecuta el algoritmo o un número fijos de pasos o evaluaciones de la función de costo ($N_{\max}$). Es simple y predecible, pero puede ser ineficiente: si se encuentra una solución óptima rápidamente, se desperdician ciclos de computación, y si $N_{\max}$ es demasiado bajo, la solución puede ser muy pobre.
- **Convergencia o Estancamiento:** Se detiene el proceso si, después de un número predefinido de iteraciones (K), la mejora en el valor de la función de costo es inferior a un umbral de tolerancia (E). Por ejemplo:

$$\left| C_{\text{mejor}}^{(i)} - C_{\text{mejor}}^{(i-k)} \right| < \epsilon$$

- **Tiempo Límite:** Se especifica un tiempo máximo de ejecución (wall-clock time o CPU time). Es crítico en aplicaciones en tiempo real o sistemas embebidos con recursos limitados, donde se requiere una respuesta rápida y garantizada, aunque sea subóptima.

Consideraciones

Diseño

Aplicaciones

Evaluación

- **Calidad Mínima Deseada:** Se finaliza al alcanzar una solución cuyo valor de costo cumpla o supere un umbral de calidad predefinido (c-umbral). Es útil cuando se conoce de antemano un valor “suficientemente bueno” a partir del cual los beneficios marginales de seguir optimizando son mínimos.
- **Combinación Híbrida:** La estrategia más robusta suele ser una combinación de los criterios anteriores (para cumplirse el tiempo límite o se alcance la calidad mínima o se detecte convergencia).
- **Selección Contextual del Criterio:**

La elección del criterio no es trivial y está profundamente ligada al dominio de aplicación:
- **Sistemas Críticos en Tiempo Real (Navegación de Drones, control de Robots):** El tiempo límite es absolutamente prioritario. La predictibilidad temporal es más importante que la optimalidad absoluta.
- **Optimización offline (Planificación de Horarios, Diseño de Circuitos, Logística Estratégica):** Se puede priorizar la calidad de la solución o la convergencia, permitiendo tiempos de ejecución largos (horas o incluso días) para encontrar soluciones near-óptimas.
- **Algoritmos Heurísticos y Metaheurísticos (Algoritmos Genéticos, enjambre de Partículas):** Es común utilizar una combinación de iteraciones máximas y criterio de convergencia para asegurar una exploración suficiente y detenerse una vez agotadas las mejoras.

Ajuste fino de parámetros

El corazón de cualquier algoritmo de planificación y optimización es la función de costo, pues determina la calidad de cada solución y orienta la búsqueda. En combinación con los criterios de parada, actúa como un mecanismo de equilibrio entre el tiempo de cómputo y calidad alcanzada, incluso en contextos donde los costos de acción pueden estimarse (Bansal et al., 2023).

Enfriamiento Simulado:

- La tasa de enfriamiento debe ser equilibrada: Una reducción demasiado rápida limita la exploración y aumenta el riesgo de óptimos locales, mientras que una demasiado lenta incrementa el coste computacional.
- La temperatura inicial debe calibrarse para permitir una adecuada diversificación en las etapas iniciales del proceso.

Gestión de Memoria (Búsqueda Tabú):

- **La longitud de la memoria es crucial:** Una duración muy corta no proporciona eficazmente los ciclos, mientras que una excesivamente larga puede restringir demasiado la exploración.
- Los criterios de aspiración introducen flexibilidad al permitir que movimientos prohibidos se acepten si conducen a soluciones excepcionales.

Diseño de vecindarios:

- **El tamaño del vecindario determina el balance entre eficiencia y capacidad de espacio:** uno pequeño es rápido, pero puede estancarse, y uno es grande es potente pero costoso. En métodos como la Búsqueda Local Iterado, la fuerza de la perturbación debe ser la mínima necesaria para escapar de la cuenta de atracción de un óptimo local sin destruir la estructura de la solución.

En consecuencia, la optimización de estos parámetros debe abordarse de manera sistemática, considerando las particularidades de cada problema y, en muchos casos, mediante procesos experimentales o métodos de optimización automática.

2.3. Aplicaciones en logística, robótica, planificación de rutas y asignación de recursos.

Los algoritmos de optimización de trayectorias constituyen herramientas fundamentales en diversos ámbitos industriales y tecnológicos debido a su capacidad para resolver problemas complejos de optimización combinatoria restringida.

Su flexibilidad y adaptabilidad permite abordar desafíos en escenarios dinámicos y con múltiples variables, ofreciendo soluciones eficientes y robustas. Las aplicaciones de la robótica en el sector logístico son muchísimas. Esto se debe a que, gracias al uso de robots, se consiguen optimizar multitud de operaciones y procesos logísticos, lo que al final repercute en la prestación de un mejor servicio y a costes más bajos. A continuación, se detallan sus principales aplicaciones.

- **Logística y Gestión de Transporte:**

Optimización de rutas de distribución (vehicle Routing Problem -VRP), donde se busca minimizar variables como la distancia total recorrida, el tiempo de entrega, el consumo de combustible o el número de vehículos utilizados, considerando restricciones de capacidad, ventajas de tiempo y demanda variables.

Gestión de flotas en tiempo real, adaptándose a imprevistos como congestiones viales, condiciones climáticas adversas y cambios en la demanda.

Asignación eficiente de pedidos y planificación de entregas mediante el uso de metaheurísticas como el Simulated Annealing o la búsqueda Tabú, las cuales permiten encontrar soluciones near-óptimas en tiempo computacionales viables.

- **Robótica y Automatización:**

Planificación de trayectorias para robots móviles y brazos robóticos en entornos estructurados o dinámicos, asegurando la evitación de obstáculos y minimizando el tiempo de movimiento o el consumo energético.

Implementación de algoritmos de búsqueda local y métodos probabilísticos para permitir la planificación en tiempo real, esencial en aplicaciones como almacenes automatizados o líneas de montaje.

- **Asignación de Recursos y Planificación Operativa:**

Diseño de horarios de horarios en contextos laborales, educativos o sanitarios, optimizando la asignación de turnos y la utilización de recursos humanos y materiales.

Balanceo de cargas de trabajo en sistemas distribuidos y centro de datos, asegurando una distribución equitativa de tareas y evitando la sobrecarga de servidores.

Coordinación de tareas en entornos de manufactura y producción, maximizando la eficiencia y reduciendo los tiempos de inactividad.

2.4. Evaluación empírica de rendimiento frente a problemas benchmark.

La evaluación empírica constituye una fase crítica en el desarrollo y validación de algoritmos de optimización, ya que permite cuantificar la eficacia, eficiencia y robustez en condiciones controladas y reproducibles.

Para ello se utilizan problemas benchmark, los cuales consisten en instancias estandarizadas y de complejidad conocida, ampliamente aceptadas por la comunidad científica para realizar comparaciones objetivas entre diferentes métodos.

Problemas Benchmark Estándar:

- **Traveling Salesman Problem (TSP):** El Problema del vendedor viajero es un problema clásico de optimización combinatoria NP-duro que sirve como referencia fundamental para evaluar algoritmos de planificación de rutas. Repositorios como TSPLIB ofrecen una amplia variedad de instancias, desde decenas hasta miles de ciudades, lo que permite analizar rigurosamente la escalabilidad y el rendimiento de un algoritmo. La búsqueda de métodos eficientes para resolver continúa siendo de gran relevancia, como lo demuestran los modernos enfoques de inteligencia artificial aplicados a su solución (Liu et al., 2022).
- **Job Shop Scheduling Problem (JSSP):** Es un problema paradigmático utilizado para evaluar la capacidad de los algoritmos para la asignación óptima de tareas en entornos industriales complejos, caracterizados por múltiples restricciones temporales y de recursos. Su principal objetivo es medir la eficiencia en la minimización del makespan (tiempo total de finalización de todos los trabajos). La complejidad inherente del JSSP lo convierte en un campo de pruebas

ideal para métodos de optimización avanzados, incluyendo aquellas estratégicas capaces de aprender y adaptarse sus políticas de planificación, lo que refleja la creciente tendencia hacia la integración de técnicas de inteligencia computacional en la gestión de operaciones (R & J, 2023).

- **Bin Packing Problem:** Se emplea para optimizar el uso del espacio en logística, buscando empaquetar ítems en contenedores minimizando su número total. Dado que en la práctica suele existir múltiples objetivos en conflicto (como equilibrar la carga entre contenedores), los algoritmos genéticos multiobjetivo (MOGA) resultan particularmente adecuados. En estos algoritmos, técnicas eficientes como el Fast Non-dominated Sorting son cruciales para comparar y clasificar soluciones frente a todos los objetivos, mejorando significativamente su rendimiento (Bahy & Musdholifah, 2022) .

Métricas de Evolución

- **Precisión:** Medida a través de la calidad de la solución obtenida (valor función de costo) en comparación con el óptimo conocido o la mejor solución documentada.
- **Eficiencia Computacional:** Evaluada mediante el tiempo de ejecución, el número de iteraciones o evaluaciones de la función de costo requeridas para alcanzar una solución satisfactoria.
- **Robustez:** Analizando mediante la desviación estándar de los resultados en múltiples ejecuciones o evaluaciones de la función de costo requeridas para alcanzar una solución satisfactoria.

Consideraciones

Diseño

Aplicaciones

Evaluación

Proceso de Evaluación:

- Ejecución de algoritmo múltiples veces sobre las mismas instancias benchmark para obtener datos estadísticamente significativos.
- Cálculo de métricas agregadas, como la media y la desviación estándar de la calidad de las soluciones y el tiempo de cómputo.
- Comparación con otros algoritmos de referencia, incluyendo metaheurísticas poblacionales (como algoritmos genéticos -GA, optimización por Enjambre de Partículas - PSO, o Colonias de Hormigas - ACO) y métodos exactos o heurísticos clásicos.

Análisis Estadístico:

- Aplicaciones de pruebas de significancia estadística, como el test t de Student para muestras independientes o el test de Wilcoxon para muestras pareadas, con el fin de determinar si las diferencias observadas en el rendimiento entre algoritmos son estadísticamente significativas.
- Utilización de diagramas de caja (boxplots) para visualizar la distribución de los resultados y detectar outliers o comportamientos anómalos.

Bibliografía

Bahy, M. B., & Musdholifah, A. (2022). Fast Non-dominated Sorting in Multi Objective Genetic Algorithm for Bin Packing Problem. IJCCS (Indonesian Journal of Computing and Cybernetics Systems), 16(1), 55-66. <https://doi.org/10.22146/ijccs.70677>

Bansal, J. C., Bajpai, P., Rawat, A., & Nagar, A. K. (2023). Advancements in the Sine Cosine Algorithm. En J. C. Bansal, P. Bajpai, A. Rawat, & A. K. Nagar (Eds.), Sine Cosine Algorithm for Optimization (pp. 87-103). Springer Nature. https://doi.org/10.1007/978-981-19-9722-8_5

Gamayunov, A., Postnikov, A., & Ferrer, G. (2023). DDPEN: Trajectory Optimisation With Sub Goal Generation Model (No. arXiv:2301.07433). arXiv. <https://doi.org/10.48550/arXiv.2301.07433>

Guilmeau, T., Chouzenoux, E., & Elvira, V. (2022). Proximal-Based Adaptive Simulated Annealing for Global Optimization. ICASSP 2022 - 2022 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP), 5453-5457. <https://doi.org/10.1109/ICASSP43922.2022.9746626>

Hu, T., Ochoa, G., & Banzhaf, W. (2023). Phenotype Search Trajectory Networks for Linear Genetic Programming (No. arXiv:2211.08516). arXiv. <https://doi.org/10.48550/arXiv.2211.08516>

Liu, G., Xu, X., Wang, F., & Tang, Y. (2022). Solving Traveling Salesman Problems Based on Artificial Cooperative Search Algorithm. Computational Intelligence and Neuroscience, 2022(1), 1008617. <https://doi.org/10.1155/2022/1008617>

R, A. K. K., & J, E. R. D. (2023). Mitigation of Make Span Time in Job Shop Scheduling Problem Using Gannet Optimization Algorithm. EAI Endorsed Transactions on Scalable Information Systems, 10(5). <https://doi.org/10.4108/eetsis.2913>

Rowold, M., Ögretmen, L., Kerbl, T., & Lohmann, B. (2022). Efficient Spatiotemporal Graph Search for Local Trajectory Planning on Oval Race Tracks. Actuators, 11(11), 319. <https://doi.org/10.3390/act11110319>

Venkateswarlu, C. (2021). A Metaheuristic Tabu Search Optimization Algorithm: Applications to Chemical and Environmental Processes. En

Consideraciones

Diseño

Aplicaciones

Evaluación

Engineering Problems—Uncertainties, Constraints and Optimization Techniques. IntechOpen. <https://doi.org/10.5772/intechopen.98240>

Weiss, E., & Kaminka, G. A. (2023). Planning with Dynamically Estimated Action Costs (No. arXiv:2206.04166). arXiv. <https://doi.org/10.48550/arXiv.2206.04166>

Xu, L., Wang, Z., Chen, X., & Lin, Z. (2022). Multi-Parking Lot and Shelter Heterogeneous Vehicle Routing Problem With Split Pickup Under Emergencies. IEEE Access, 10, 36073-36090. <https://doi.org/10.1109/ACCESS.2022.3163715>